

FECAP

Ciência da Computação – 6º Semestre

Teoria da Computação e Linguagens Formais
Entrega 1

Integrantes:

Bernardo Seijas Cavalcante
Carlos Roberto Santos Latorre
Felipe Wakasa Klabunde
Stephany Aliyah Guimarães Eurípedes de Paula

Introdução

No contexto do projeto Smart Tennis Arena, desenvolvemos o TennisSync, um treinador interativo de tênis que utiliza visão computacional e gamificação para auxiliar o jogador na execução dos movimentos. Durante o treinamento, o sistema apresenta um movimento de referência na tela e aguarda o acionamento de um pedal para iniciar a tentativa. Após a execução, o movimento realizado é analisado e comparado com os parâmetros definidos para o exercício. Com base nessa avaliação, o jogador recebe uma pontuação e, quando necessário, orientações corretivas por meio de feedback visual e por voz. Para representar esse funcionamento, foi utilizada uma Máquina de Estados Finitos, relacionando o comportamento do TennisSync aos conceitos de autômatos, linguagens formais, expressões regulares e gramáticas estudados na disciplina.

1. Identificação dos Estados do Sistema

Para representar o funcionamento do projeto, foram definidos os principais estados pelos quais o sistema pode passar durante uma sessão de treinamento. Cada estado corresponde a uma etapa específica, desde a inicialização dos componentes até a análise do movimento, apresentação do feedback e encerramento da sessão.

Os estados identificados foram utilizados na construção do diagrama e da tabela de transições, permitindo representar de forma organizada o comportamento do sistema.

- **q0 – Inicializando:** é o estado inicial do sistema. Nessa etapa são iniciados o software, a câmera, o MediaPipe e a comunicação com o Arduino.
- **q1 – Pronto:** indica que a inicialização terminou corretamente e que o sistema está disponível para iniciar o treinamento.
- **q2 – Demonstração de Movimento:** a tela apresenta ao jogador o movimento de referência que deverá ser executado.
- **q3 – Aguardando:** depois da demonstração, o sistema aguarda a ação do jogador. O pedal pode ser acionado para iniciar a tentativa, e a sessão também pode ser pausada ou encerrada nesse estado.
- **q4 – Captura:** a câmera registra a execução do jogador e o MediaPipe identifica os pontos de referência do corpo.
- **q5 – Análise:** os dados capturados são processados e comparados com os parâmetros definidos para o movimento esperado.
- **q6 – Correto:** indica que a execução foi considerada correta de acordo com os critérios definidos. A partir desse estado, o treino pode avançar para o próximo movimento ou ser encerrado.
- **q7 – Erro de Movimento:** indica que foi encontrado um erro na execução do movimento. Esse estado representa um erro do jogador, e não uma falha técnica do sistema.

- **q8 – Feedback:** o sistema apresenta ao jogador a orientação necessária para corrigir o movimento, utilizando a tela, os LEDs e o retorno por voz.
- **q9 – Pausado:** representa uma interrupção temporária da sessão. Ao retomar, o sistema volta para a demonstração do movimento.
- **q10 – Encerrado:** é o estado final da sessão de treinamento.
- **qE – Falha:** representa uma falha técnica, como problema na câmera, perda de comunicação com o Arduino ou erro crítico de software ou hardware.

A distinção entre **q7 – Erro de Movimento** e **qE – Falha** é importante para a modelagem. O estado **q7** representa uma tentativa em que o movimento foi executado fora dos parâmetros definidos, enquanto **qE** indica um problema técnico que compromete o funcionamento do sistema.

2. Definição dos Eventos e Gatilhos de Transição (Entradas)

Os eventos são responsáveis pelas mudanças entre os estados do TennisSync. Eles podem representar ações do jogador, comandos do sistema ou condições identificadas durante o treinamento.

Entre esses eventos estão o acionamento do pedal, a conclusão da captura do movimento, o resultado da análise e a ocorrência de falhas técnicas. Cada evento provoca uma transição específica, conforme definido na máquina de estados.

- **sistema_ok:** a inicialização foi concluída corretamente.
- **iniciar:** inicia a sessão de treinamento.
- **fim_demo:** indica que a demonstração do movimento terminou.
- **pedal:** representa o acionamento do pedal pelo jogador.
- **fim_captura:** indica que a captura do movimento foi concluída.
- **resultado_ok:** indica que o movimento foi considerado correto.
- **resultado_erro:** indica que foi identificado um erro na execução do movimento.
- **emitir_feedback:** inicia a apresentação do feedback corretivo.
- **fim_feedback:** indica que o feedback terminou e o movimento poderá ser demonstrado novamente.
- **proximo:** faz o sistema seguir para a demonstração do próximo movimento ou repetição.
- **pausar:** interrompe temporariamente a sessão.
- **retomar:** continua a sessão após a pausa e retorna à demonstração.
- **encerrar:** finaliza a sessão de treinamento.
- **falha:** indica uma falha técnica no sistema.
- **reset:** reinicia o sistema após uma falha.

3. Diagrama de Estados

O diagrama de estados apresenta o funcionamento do sistema, ilustrando as principais etapas do treinamento e os eventos responsáveis pelas mudanças entre elas. Essa representação gráfica permite acompanhar o fluxo completo da aplicação, desde a inicialização até a análise do movimento, contemplando os possíveis resultados, as rotinas de feedback, as pausas e o encerramento da sessão.

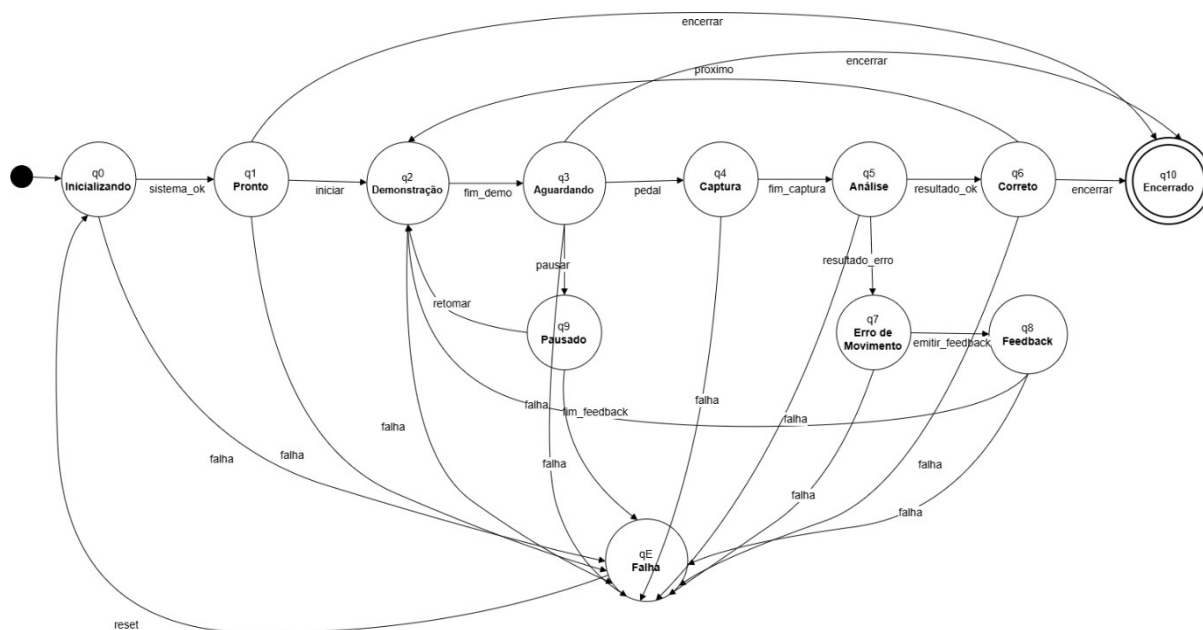


Figura 1 – Diagrama de Estados do TennisSync

O fluxo começa em **q0 (Inicializando)**. Quando a inicialização termina corretamente, o evento sistema_ok leva o sistema para **q1 (Pronto)**. Ao receber o evento iniciar, o sistema passa para **q2 (Demonstração de Movimento)**. Depois do fim da demonstração, o evento fim_demo leva para **q3 (Aguardando)**. Quando o jogador pressiona o pedal, ocorre a transição para **q4 (Captura)**. Ao finalizar a captura, o evento fim_captura leva o sistema para **q5 (Análise)**. Na análise existem dois resultados principais. Se o movimento estiver correto, resultado_ok leva para **q6 (Correto)**. Se for identificado um erro na execução, resultado_erro leva para **q7 (Erro de Movimento)**.

Quando existe erro no movimento, emitir_feedback leva para **q8 (Feedback)**. Após fim_feedback, o sistema retorna para **q2 (Demonstração de Movimento)**, permitindo uma nova tentativa. No estado **q6 (Correto)**, o evento proximo retorna para **q2**, iniciando a demonstração do próximo movimento. O evento encerrar leva para **q10 (Encerrado)**. A pausa ocorre a partir de **q3** por meio do evento pausar. Em **q9 (Pausado)**, o evento retomar faz o sistema voltar para **q2**. O diagrama também permite o encerramento a partir de **q1**, **q3** e **q6**. Em caso de falha técnica, os estados

de funcionamento podem levar para **qE (Falha)**. O evento reset faz o sistema retornar para **q0 (Inicializando)**.

3.1 Definição Formal do Autômato

O comportamento do projeto foi representado por uma máquina de estados finitos determinística, considerando os eventos permitidos em cada etapa do treinamento. Nesse modelo, para um mesmo estado e evento de entrada, existe no máximo um próximo estado definido.

A representação formal utilizada é:

$$M = (Q, \Sigma, \delta, q_0, F)$$

- **Q**: conjunto de estados do sistema.
- **Σ** : alfabeto formado pelos eventos de entrada.
- **δ** : função de transição entre os estados.
- **q_0** : estado inicial.
- **F**: conjunto de estados finais.

A função de transição é parcial, pois nem todos os eventos estão definidos para todos os estados. As combinações sem transição prevista aparecem como células vazias na matriz de transições.

Aplicando esta estrutura aos dados do nosso projeto, temos os seguintes elementos:

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9, q_{10}, q_E\}$$

$$q_0 = q_0$$

$$F = \{q_{10}\}$$

A função de transição determina qual será o próximo estado a partir do estado atual e do evento recebido. Por exemplo:

$$\delta(q_3, \text{pedal}) = q_4$$

Isso significa que, quando o sistema está no estado **q3 – Aguardando** e recebe o evento **pedal**, passa para o estado **q4 – Captura**.

4. Tabela de Transições

A tabela de transições apresenta as mudanças de estado que podem ocorrer durante o funcionamento do sistema. Inicialmente, as transições são representadas em formato de matriz, com os estados nas linhas e os eventos nas colunas. Cada célula preenchida indica o próximo estado, enquanto as células vazias representam combinações para as quais não foi definida uma transição.

δ / Estado	o	i	d	p	c	k	e	f	t	x	s	r	z	u	b
→ q0	q1													qE	
q1		q2											q10	qE	
q2			q3											qE	
q3				q4							q9		q10	qE	
q4					q5									qE	
q5						q6	q7							qE	
q6										q2			q10	qE	
q7								q8						qE	
q8									q2					qE	
q9												q2		qE	
★ q10															
qE															q0

As mesmas transições são exibidas na tabela a seguir, utilizando os nomes completos dos eventos para facilitar a leitura.

Estado Atual	Evento de Entrada	Próximo Estado
q0	sistema_ok	q1
q1	iniciar	q2
q1	encerrar	q10
q2	fim_demo	q3
q3	pedal	q4
q3	pausar	q9
q3	encerrar	q10
q4	fim_captura	q5
q5	resultado_ok	q6
q5	resultado_erro	q7
q6	proximo	q2
q6	encerrar	q10
q7	emitir_feedback	q8
q8	fim_feedback	q2
q9	retomar	q2
q0 a q9	falha	qE
qE	reset	q0

Na última linha de falha, a indicação q0 a q9 resume as transições para qE que aparecem no diagrama. Assim, não é necessário repetir uma linha de falha para cada estado operacional.

5. Linguagem do Sistema

A linguagem formal é composta por sequências de eventos que representam o funcionamento do sistema. Para sua construção, cada evento recebeu um símbolo específico, formando o alfabeto utilizado na modelagem, conforme apresentado na tabela a seguir.

Símbolo	Evento
o	sistema_ok
i	iniciar
d	fim_demo
p	pedal
c	fim_captura
k	resultado_ok
e	resultado_erro
f	emitir_feedback
t	fim_feedback
x	proximo
s	pausar
r	retomar
z	encerrar
u	falha
b	reset

A partir dos símbolos apresentados, o alfabeto do sistema é definido por:

$$\Sigma = \{o, i, d, p, c, k, e, f, t, x, s, r, z, u, b\}$$

A linguagem do projeto é formada pelas sequências desses símbolos que respeitam as transições permitidas pelo autômato.

$$L(M) = \{w \in \Sigma^* \mid \delta^*(q0, w) = q10\}$$

Nessa expressão, w representa uma sequência de eventos e δ^* representa a função de transição aplicada a essa sequência. Uma cadeia é considerada aceita quando parte do estado inicial **q0** e chega ao estado final **q10**, respeitando as transições definidas.

5.1 Exemplos de Sequências Válidas

Um treino em que o jogador acerta na primeira tentativa pode ser representado por:

$$w_1 = o \cdot i \cdot d \cdot p \cdot c \cdot k \cdot z$$

A sequência corresponde a: inicialização correta, início da sessão, fim da demonstração, acionamento do pedal, fim da captura, movimento correto e encerramento.

Um exemplo em que o jogador erra a primeira tentativa e depois acerta é:

$$w_2 = o \cdot i \cdot d \cdot p \cdot c \cdot e \cdot f \cdot t \cdot d \cdot p \cdot c \cdot k \cdot z$$

Nesse caso, depois do erro o sistema fornece feedback, demonstra novamente o movimento e permite uma nova tentativa.

Também é possível representar uma pausa durante o treinamento:

$$w_3 = o \cdot i \cdot d \cdot s \cdot r \cdot d \cdot p \cdot c \cdot k \cdot z$$

Nesse exemplo, a pausa ocorre em **q3 – Aguardando**. Ao retomar, o sistema volta para **q2 – Demonstração de Movimento** e depois segue normalmente.

5.2 Exemplo de Sequência Inválida

$$w_4 = o \cdot i \cdot k \cdot z$$

Essa sequência não é válida, porque **resultado_ok** não pode acontecer logo após iniciar. Antes disso, o sistema precisa passar pela demonstração, espera pelo pedal, captura e análise.

6. Expressão Regular

A expressão regular abaixo representa o fluxo principal de um treinamento concluído com sucesso. Para facilitar a leitura, ela não inclui pausa, falha técnica nem encerramento antecipado.

$$E = o \cdot i \cdot d \cdot (p \cdot c \cdot e \cdot f \cdot t \cdot d)^* \cdot p \cdot c \cdot k \cdot (x \cdot d \cdot (p \cdot c \cdot e \cdot f \cdot t \cdot d)^* \cdot p \cdot c \cdot k)^* \cdot z$$

O trecho $(p \cdot c \cdot e \cdot f \cdot t \cdot d)^*$ representa zero ou mais tentativas com erro. Nessas tentativas, o jogador aciona o pedal, o sistema captura e analisa o movimento, identifica o erro, apresenta o feedback e volta para a demonstração.

Depois disso, $p \cdot c \cdot k$ representa uma tentativa correta. O trecho iniciado por x permite repetir esse processo para o próximo movimento. Por fim, z representa o encerramento da sessão.

7. Gramática

A gramática regular foi construída com base nas transições definidas para a máquina de estados. Nessa representação, as variáveis correspondem aos estados e os símbolos terminais representam os eventos do sistema.

$$G = (V, \Sigma, R, S_0)$$

Cada elemento da definição representa:

- **G**: a gramática utilizada para representar as sequências do TennisSync.
- **V**: o conjunto de variáveis, também chamadas de símbolos não terminais.
- **Σ** : o conjunto de símbolos terminais. Neste trabalho, eles representam os eventos do sistema.
- **R**: o conjunto de regras de produção que define como as sequências podem ser formadas.
- **S_0** : o símbolo inicial da gramática, associado ao estado inicial q_0 .

O conjunto de variáveis é:

$$V = \{S_0, Q_1, Q_2, Q_3, Q_4, Q_5, Q_6, Q_7, Q_8, Q_9, Q_{10}, Q_E\}$$

O conjunto de símbolos terminais é:

$$\Sigma = \{o, i, d, p, c, k, e, f, t, x, s, r, z, u, b\}$$

As regras de produção são:

$$\begin{aligned} S_0 &\rightarrow oQ_1 \mid uQ_E \\ Q_1 &\rightarrow iQ_2 \mid zQ_{10} \mid uQ_E \\ Q_2 &\rightarrow dQ_3 \mid uQ_E \\ Q_3 &\rightarrow pQ_4 \mid sQ_9 \mid zQ_{10} \mid uQ_E \\ Q_4 &\rightarrow cQ_5 \mid uQ_E \\ Q_5 &\rightarrow kQ_6 \mid eQ_7 \mid uQ_E \\ Q_6 &\rightarrow xQ_2 \mid zQ_{10} \mid uQ_E \\ Q_7 &\rightarrow fQ_8 \mid uQ_E \\ Q_8 &\rightarrow tQ_2 \mid uQ_E \\ Q_9 &\rightarrow rQ_2 \mid uQ_E \\ Q_E &\rightarrow bS_0 \\ Q_{10} &\rightarrow \varepsilon \end{aligned}$$

Por exemplo, $Q_3 \rightarrow pQ_4$ indica que o evento p, correspondente ao pedal, leva o sistema do estado de espera para o estado de captura.

As regras $Q_5 \rightarrow kQ_6$ e $Q_5 \rightarrow eQ_7$ representam os dois resultados possíveis da análise: movimento correto ou erro na execução.

Quando existe erro, $Q_7 \rightarrow fQ_8$ leva para o feedback. Em seguida, $Q_8 \rightarrow tQ_2$ faz o sistema voltar para a demonstração do movimento.

A regra $Q_9 \rightarrow rQ_2$ representa a retomada após uma pausa. Já $Q_E \rightarrow bS_0$ representa o reset após uma falha.

Por fim, $Q_{10} \rightarrow \varepsilon$ indica que a sequência termina quando o sistema chega ao estado q_{10} – Encerrado. O símbolo ε representa a palavra vazia.

8. Justificativa da Modelagem

A Máquina de Estados Finitos foi escolhida porque o TennisSync possui etapas bem definidas. Durante uma sessão, o sistema passa por diferentes situações, como inicializar os componentes, demonstrar o movimento, aguardar o jogador, capturar os dados e analisar a execução.

As mudanças entre esses estados acontecem a partir de eventos relacionados ao próprio funcionamento do projeto. Um exemplo ocorre em **q3 (Aguardando)**: quando o jogador pressiona o pedal conectado ao Arduino, o sistema passa para **q4 (Captura)**.

Depois da captura, o TennisSync entra em **q5 (Análise)**. Nesse estado, os dados obtidos pelo MediaPipe são utilizados pelas regras definidas no projeto para verificar se o movimento realizado está dentro dos parâmetros esperados.

A análise pode levar para **q6 (Correto)** ou **q7 (Erro de Movimento)**. Caso seja identificado um erro na execução, o sistema segue para **q8 (Feedback)** e, quando essa etapa termina, retorna para **q2**, onde o movimento é demonstrado novamente.

A modelagem também permite diferenciar os erros de execução das falhas técnicas. O estado **q7** representa um erro cometido pelo jogador durante o exercício, enquanto **qE (Falha)** corresponde a problemas no funcionamento do hardware ou software. Essa separação permite que cada situação receba o tratamento adequado.

Além disso, o estado **q9 (Pausado)** representa uma interrupção temporária do treinamento, enquanto **q10 (Encerrado)** indica o término da sessão.

A linguagem formal, a expressão regular e a gramática complementam o diagrama ao representar as sequências de eventos permitidas durante o funcionamento do TennisSync. Dessa maneira, a modelagem relaciona o comportamento do projeto aos conceitos estudados na disciplina e pode servir como referência para as próximas etapas de desenvolvimento.

Conclusão

A modelagem apresentada descreve o funcionamento principal do TennisSync desde a inicialização até o encerramento de uma sessão.

O diagrama e a tabela de transições mostram como o sistema reage aos eventos durante o treinamento. Já a linguagem, a expressão regular e a gramática representam essas mesmas regras utilizando os conceitos trabalhados na disciplina. Além de atender à proposta da entrega, essa modelagem pode servir como referência durante o desenvolvimento do projeto, principalmente na integração entre Arduino, pedal, câmera, MediaPipe, LEDs e feedback por voz.